

Sistema de Gestión de Proyectos y Tareas Colaborativo

Autor: Mario González Giménez

Ciclo: Desarrollo de Aplicaciones Web (DAW) — 2.º curso

Centro: IES La Vereda

Tutor: Victor Pla Soler

Curso: 2025/2026

Fecha: Mayo 2026

Resumen del proyecto

Aplicación web colaborativa de gestión de proyectos y tareas basada en metodología Kanban. Desarrollada con Angular 20 en el frontend, Spring Boot en el backend y PostgreSQL como base de datos. Permite a equipos crear proyectos, definir etapas, gestionar tareas y colaborar con control de roles. Incluye interfaz responsiva con soporte de modo claro y oscuro.

Resumen

Resumen (español)

Esta memoria describe el desarrollo de una aplicación web de gestión de proyectos y tareas colaborativa. La aplicación permite a los usuarios registrarse, crear proyectos, añadir miembros y gestionar tareas organizadas en etapas mediante un tablero Kanban. El sistema está desarrollado con Spring Boot en el backend, Angular 20 en el frontend y PostgreSQL como base de datos.

Resum (valencià)

Aquesta memòria descriu el desenvolupament d'una aplicació web de gestió de projectes i tasques col·laborativa. L'aplicació permet als usuaris registrar-se, crear projectes, afegir membres i gestionar tasques organitzades en etapes mitjançant un tauler Kanban. El sistema està desenvolupat amb Spring Boot al backend, Angular 20 al frontend i PostgreSQL com a base de dades.

Abstract (English)

This report describes the development of a collaborative web application for project and task management. The application allows users to register, create projects, add members and manage tasks organised in stages using a Kanban board. The system is built with Spring Boot on the backend, Angular 20 on the frontend and PostgreSQL as the database.

Índice general

Resumen	2
1. Introducción	7
1.1. Módulos del ciclo implicados	7
1.2. Descripción del proyecto	8
2. Justificación	9
3. Estudio previo	10
3.1. Análisis de soluciones existentes	10
3.2. Tecnologías investigadas	11
4. Objetivos	12
5. Planificación	13
6. Análisis de requisitos	14
6.1. Requisitos funcionales	14
6.2. Requisitos no funcionales	15
6.3. Casos de uso	15
6.4. Roles y permisos	16
7. Diseño del sistema	18
7.1. Arquitectura	18
7.2. Modelo de datos	19
7.3. Diseño de la API REST	19
7.4. Diseño de la interfaz	20
8. Implementación	21
8.1. Stack tecnológico	21
8.2. Estructura del proyecto	21
8.3. Decisiones técnicas destacadas	22
9. Pruebas	24
9.1. Pruebas unitarias del backend	24
9.2. Pruebas unitarias del frontend	25
9.3. Integración y despliegue continuos	25
9.4. Pruebas funcionales del frontend	25
10. Manual de despliegue	27
10.1. Requisitos previos	27
10.2. Con Docker (recomendado)	27
10.3. Sin Docker	27
10.4. Frontend	28

11. Manual de usuario	29
11.1. Registro e inicio de sesión	29
11.2. Dashboard	30
11.3. Tablero Kanban	30
11.4. Gestión de miembros	31
12. Análisis del producto (FOL)	32
12.1. Descripción del proyecto como producto	32
12.2. Análisis DAFO	33
12.3. Análisis de mercado	33
12.4. Propuesta de mejora y evolución futura	36
13. Conclusiones	37
13.1. Grado de consecución de objetivos	37
13.2. Problemas encontrados	37
13.3. Valoración personal	37
14. Repositorio y software utilizado	38
14.1. Repositorio GitHub	38
14.2. Software utilizado	39
Bibliografía	40

Índice de figuras

5.1. Diagrama de Gantt — Planificación del proyecto	13
6.1. Diagrama de casos de uso	15
7.1. Diagrama de arquitectura del sistema	18
7.2. Diagrama entidad-relación	19
11.1. Formulario de login. Elaboración propia.	29
11.2. Dashboard con lista de proyectos. Elaboración propia.	30
11.3. Tablero Kanban con columnas y tareas. Elaboración propia.	30
11.4. Página de miembros del proyecto. Elaboración propia.	31

Índice de tablas

1.1. Módulos del ciclo y RAs aplicados	7
3.1. Comparativa de herramientas de gestión de proyectos	10
3.2. Tecnologías investigadas fuera del plan de estudios	11
4.1. Objetivos específicos del proyecto	12
5.1. Fases de desarrollo del proyecto	13
6.1. Requisitos funcionales	14
6.2. Requisitos no funcionales	15
6.3. Permisos por rol	17
7.1. Endpoints de la API REST	19
8.1. Stack tecnológico y justificación de las elecciones	21
9.1. Cobertura de pruebas unitarias del backend (105 tests, 100% cobertura)	24
9.2. Pruebas unitarias del frontend (Vitest)	25
9.3. Pruebas funcionales del frontend	26
10.1. Requisitos de software para el despliegue	27
12.1. Análisis DAFO del producto	33
12.2. Comparativa de competidores en el mercado	34
12.3. Hoja de ruta de mejoras del producto	36
14.1. Estructura del repositorio	38
14.2. Software utilizado en el desarrollo	39

Capítulo 1

Introducción

Este proyecto consiste en el desarrollo de una aplicación web de gestión de proyectos y tareas colaborativa, inspirada en herramientas como Trello o Jira. La aplicación permite a un equipo de usuarios crear proyectos, organizarlos en etapas personalizadas y gestionar tareas dentro de cada etapa mediante un tablero Kanban.

La aplicación sigue una arquitectura cliente-servidor desacoplada: el backend expone una API REST desarrollada con Spring Boot, y el frontend es una Single Page Application (SPA) construida con Angular 20. La comunicación entre ambas capas se realiza exclusivamente a través de JSON.

1.1. Módulos del ciclo implicados

El proyecto integra conocimientos y capacidades desarrollados a lo largo del ciclo formativo de Desarrollo de Aplicaciones Web. En la siguiente tabla se recogen los módulos y los resultados de aprendizaje (RAs) que se aplican directamente en el proyecto.

Tabla 1.1: Módulos del ciclo y RAs aplicados

Módulo	RA	Aplicación en el proyecto
Desarrollo web en entorno servidor	1	Selección de arquitectura REST con Spring Boot
Desarrollo web en entorno servidor	5	Separación de presentación y lógica de negocio (capas Controller, Service, Repository)
Desarrollo web en entorno servidor	6	Acceso a PostgreSQL con Spring Data JPA, integridad referencial
Desarrollo web en entorno servidor	7	API REST consumible por cualquier cliente HTTP
Diseño de interfaces web	1	Planificación de la interfaz: wireframes, paleta, tipografía
Diseño de interfaces web	2	Sistema de variables CSS (tokens) para estilos homogéneos en toda la aplicación
Diseño de interfaces web	5	Interfaz responsiva adaptada a móvil y escritorio con criterios de accesibilidad
Diseño de interfaces web	6	Patrones de usabilidad: feedback visual, confirmaciones, notificaciones toast
Bases de datos	2	Diseño del modelo relacional: users, project, stage, task, user_project
Bases de datos	3	Consultas JPA para listado y filtrado de proyectos, etapas y tareas

Módulo	RA	Aplicación en el proyecto
Bases de datos	4	Operaciones de inserción, actualización y borrado desde los servicios
Programación	4	Diseño orientado a objetos con principios SOLID en el backend
Programación	7	Uso de características avanzadas de Java 21 (streams, lambdas, Comparator)
Lenguaje de marcas	2	Maquetación HTML semántica y uso de TypeScript en el frontend
Lenguaje de marcas	3	Manipulación dinámica del DOM mediante Angular y signals

1.2. Descripción del proyecto

El sistema permite a los usuarios:

- Registrarse e iniciar sesión con email y contraseña.
- Crear y gestionar proyectos, invitando a otros usuarios con rol asignado.
- Definir las etapas del flujo de trabajo de cada proyecto (columnas del tablero).
- Crear, editar, mover y eliminar tareas dentro de las etapas.
- Acceder al tablero desde cualquier dispositivo gracias al diseño responsivo.

Entre las tecnologías investigadas fuera del plan de estudios destacan: la API de signals de Angular 20, el sistema de *scroll snap* CSS para navegación por columnas en móvil, y las técnicas de CSS como *scroll shadow* y *CSS mask* para iconos.

Capítulo 2

Justificación

La gestión visual del trabajo es una necesidad real en cualquier equipo de desarrollo. Herramientas como Trello, Asana o Jira son ampliamente utilizadas en la industria, pero su implementación interna raramente se estudia en profundidad. Desarrollar una herramienta similar desde cero permite aplicar de forma integrada los conocimientos adquiridos durante el ciclo:

- Diseño e implementación de una API REST con Spring Boot.
- Desarrollo de una SPA con Angular y patrones reactivos modernos.
- Modelado y gestión de una base de datos relacional con PostgreSQL.
- Gestión del proyecto con control de versiones Git.
- Contenerización de servicios con Docker.

Además, el proyecto refleja un caso de uso real: la colaboración en equipo con roles, permisos y visibilidad del estado del trabajo.

Capítulo 3

Estudio previo

3.1. Análisis de soluciones existentes

Antes de definir el alcance del proyecto se analizaron las herramientas de gestión de proyectos más utilizadas, con el objetivo de identificar sus puntos fuertes, sus limitaciones y el hueco que una herramienta propia podría cubrir.

Tabla 3.1: Comparativa de herramientas de gestión de proyectos

Herramienta	Puntos fuertes	Limitaciones	Precio base
Trello	Interfaz muy simple, Kanban puro, plan gratuito generoso	Sin subtareas, sin informes, automatizaciones limitadas en gratuito	Gratis / 5\$ usuario/mes
Asana	Vistas múltiples (lista, tablero, cronograma), buena gestión de dependencias	Curva de aprendizaje media, funciones avanzadas solo en plan de pago	Gratis hasta 15 usuarios / 10\$
Jira	Muy configurable, integración nativa con GitHub/GitLab, gestión de sprints	Complejo para equipos pequeños, lento en proyectos grandes	Gratis hasta 10 usuarios / 7\$
Notion	Todo-en-uno (docs + tareas + wiki), flexible	No está especializado en gestión de proyectos, sin vistas de Gantt nativas	Gratis / 8\$
Linear	Muy rápido, orientado a desarrollo de software, buena UX	Poco flexible fuera del contexto de desarrollo, sin plan gratuito permanente	8\$ usuario/mes

Conclusiones del análisis

Las herramientas existentes cubren bien el mercado enterprise y los equipos medianos, pero presentan complejidad o coste excesivos para equipos pequeños y entornos académicos. La oportunidad identificada es una herramienta:

- Con la simplicidad visual de Trello pero con gestión de roles por proyecto.
- De código abierto, desplegable en cualquier servidor sin coste de licencia.

- Construida con tecnologías modernas y estándar del sector (Angular, Spring Boot), lo que facilita su comprensión y extensión.

3.2. Tecnologías investigadas

Tabla 3.2: Tecnologías investigadas fuera del plan de estudios

Tecnología / Técnica	Descripción y motivación
Angular Signals (API nativa)	Sistema de reactividad introducido en Angular 16 y estabilizado en Angular 17–20. Sustituye a los patrones basados en RxJS en la mayoría de los componentes, simplificando el código y mejorando el rendimiento.
CSS Scroll Snap	Propiedad CSS que permite que el scroll se «enclave» en puntos concretos. Usada para la navegación columna a columna en móvil sin ninguna librería JavaScript.
CSS custom properties (tokens)	Sistema de variables CSS organizado en un archivo de tokens para gestionar la paleta de colores completa en modo claro y oscuro desde un único punto.
CSS mask	Técnica para aplicar iconos SVG a través de CSS sin ensuciar el HTML con SVG inline, manteniendo la adaptabilidad de color mediante <code>currentColor</code> .
<code>color-scheme</code>	Propiedad CSS que instruye al navegador a renderizar los controles nativos (inputs, selects) en modo claro u oscuro automáticamente.

Capítulo 4

Objetivos

Objetivo general

Desarrollar una aplicación web funcional que permita gestionar proyectos y tareas colaborativas mediante un tablero Kanban.

Objetivos específicos

Tabla 4.1: Objetivos específicos del proyecto

#	Objetivo	Logrado
OE-01	Implementar registro y autenticación de usuarios	✓
OE-02	CRUD de proyectos con gestión de miembros	✓
OE-03	Etapas Kanban configurables por proyecto	✓
OE-04	CRUD completo de tareas	✓
OE-05	Interfaz Angular con tablero Kanban interactivo	✓
OE-06	Mover tareas entre columnas	✓
OE-07	Edición y eliminación de tareas con confirmación	✓
OE-08	Interfaz responsiva para escritorio y móvil	✓
OE-09	Soporte de modo claro y oscuro automático	✓
OE-10	Gestión de miembros con roles por proyecto	✓

Capítulo 5

Planificación

El desarrollo se ha dividido en fases incrementales, cada una con un objetivo claro y verificable. Cada fase produce commits atómicos en el repositorio Git, lo que permite trazar la evolución del proyecto con precisión.

Tabla 5.1: Fases de desarrollo del proyecto

Fase	Descripción	Estado
Backend	Modelos JPA, servicios, repositorios, API REST completa	✓
Setup	Modelos de dominio TypeScript, servicios API, configuración	✓
Fase 2	AuthService, interceptor HTTP, guard de rutas	✓
Fase 3	Layout principal, navbar, rutas con lazy loading	✓
Fase 4	Formularios de login y registro con manejo de errores	✓
Fase 5	Dashboard con listado de proyectos y creación reactiva	✓
Fase 6	Tablero Kanban: columnas y tarjetas de tarea	✓
Fase 6b	Movimiento de tareas entre columnas	✓
Fase 7	Modal de edición de tareas y diálogo de confirmación	✓
Fase 8	Notificaciones toast	✓
UI avanzada	Diseño visual, responsividad, dark/light mode, gestión de etapas	✓

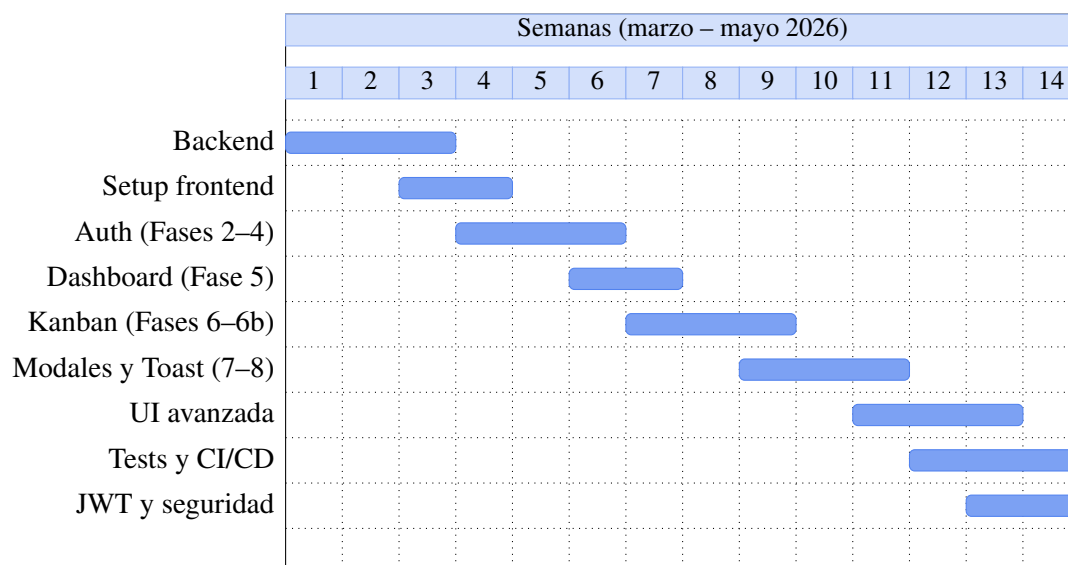


Figura 5.1: Diagrama de Gantt — Planificación del proyecto

Capítulo 6

Análisis de requisitos

6.1. Requisitos funcionales

Tabla 6.1: Requisitos funcionales

Código	Requisito
RF-01	El sistema debe permitir registrar nuevos usuarios con nombre, email y contraseña
RF-02	El sistema debe permitir iniciar sesión con email y contraseña
RF-03	El sistema debe permitir crear, editar y eliminar proyectos
RF-04	El sistema debe permitir añadir y eliminar miembros de un proyecto con un rol asignado
RF-05	El sistema debe permitir crear, editar, renombrar y eliminar etapas dentro de un proyecto
RF-06	El sistema debe permitir reordenar las etapas de un proyecto
RF-07	El sistema debe permitir crear, editar y eliminar tareas dentro de una etapa
RF-08	El sistema debe permitir mover una tarea a otra etapa
RF-09	El sistema debe mostrar un tablero Kanban con las etapas y tareas del proyecto
RF-10	El sistema debe mostrar el listado de proyectos del usuario autenticado
RF-11	El sistema debe confirmar las acciones destructivas antes de ejecutarlas
RF-12	El sistema debe mostrar notificaciones de éxito o error tras cada acción

6.2. Requisitos no funcionales

Tabla 6.2: Requisitos no funcionales

Código	Requisito
RNF-01	La interfaz debe ser responsiva y funcionar correctamente en escritorio y móvil
RNF-02	La interfaz debe adaptarse automáticamente al modo claro u oscuro del sistema
RNF-03	El tiempo de respuesta de la API no debe superar los 2 segundos en condiciones normales
RNF-04	El frontend debe cargarse como SPA sin recargas completas de página
RNF-05	El código debe seguir principios SOLID y estar organizado por capas
RNF-06	La aplicación debe ser desplegable con Docker en cualquier entorno

6.3. Casos de uso

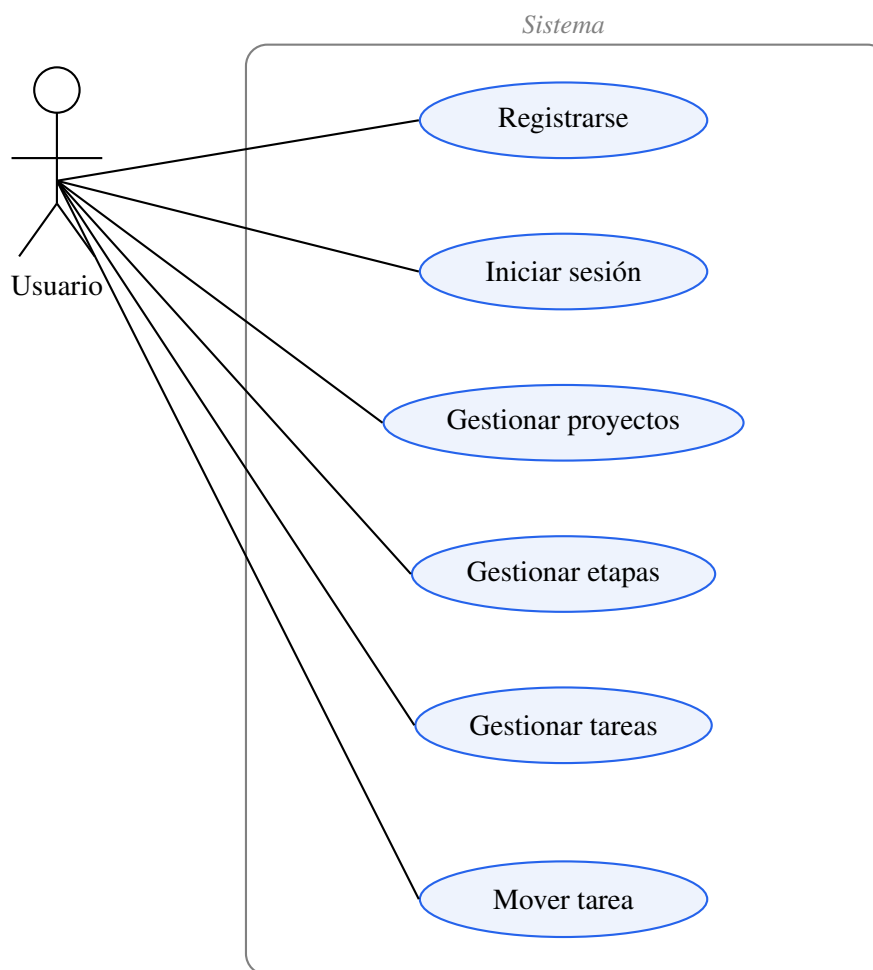


Figura 6.1: Diagrama de casos de uso

CU-01 — Iniciar sesión

Actor: Usuario no autenticado

Precondición: el usuario existe en el sistema

Flujo: el usuario introduce email y contraseña → el sistema verifica las credenciales → redirige al dashboard

Postcondición: el usuario queda autenticado y su sesión se persiste en localStorage

CU-02 — Crear tarea

Actor: Usuario autenticado miembro del proyecto

Precondición: existe al menos una etapa en el proyecto

Flujo: el usuario pulsa «Añadir tarea» en una columna → introduce el título → la tarea aparece en esa columna

Postcondición: la tarea queda registrada en base de datos asociada a la etapa

CU-03 — Mover tarea entre columnas

Actor: Usuario autenticado miembro del proyecto

Precondición: existen al menos dos etapas y la tarea está creada

Flujo: el usuario selecciona la columna destino en el modal de edición → la tarea desaparece de la columna actual y aparece en la destino

Postcondición: la tarea queda vinculada a la nueva etapa en base de datos

6.4. Roles y permisos

El sistema define tres roles por proyecto. La siguiente tabla recoge qué acciones puede realizar cada rol:

Tabla 6.3: Permisos por rol

Acción	OWNER	ADMIN	MEMBER
Ver tablero y tareas	✓	✓	✓
Crear tarea	✓	✓	✓
Editar tarea	✓	✓	✓
Mover tarea de columna	✓	✓	✓
Eliminar tarea	✓	✓	✓
Renombrar proyecto	✓	✓	—
Crear / editar / eliminar etapa	✓	✓	—
Reordenar etapas	✓	✓	—
Añadir miembro	✓	✓	—
Cambiar rol de miembro	✓	✓	—
Eliminar miembro	✓	✓	—
Eliminar proyecto	✓	—	—
Abandonar proyecto	—	✓	✓

El propietario (*OWNER*) no puede abandonar el proyecto; debe eliminarlo o transferir la propiedad previamente.

Capítulo 7

Diseño del sistema

7.1. Arquitectura

La aplicación sigue una arquitectura cliente-servidor en tres capas:

- **Capa de presentación:** SPA Angular que consume la API y renderiza la interfaz en el navegador.
- **Capa de negocio:** API REST Spring Boot que gestiona la lógica de negocio y expone los endpoints.
- **Capa de datos:** PostgreSQL 16 gestionado a través de Spring Data JPA / Hibernate.

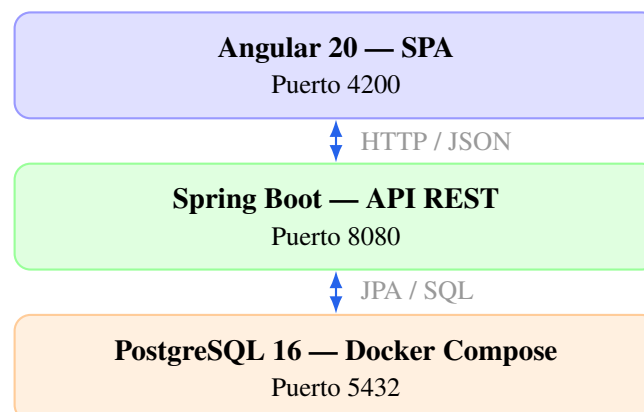


Figura 7.1: Diagrama de arquitectura del sistema

La comunicación entre frontend y backend se realiza exclusivamente mediante HTTP con respuestas JSON. Las peticiones autenticadas incluyen la cabecera `Authorization: Bearer <token>` con un token JWT firmado que identifica al usuario. El token se genera en el login y es validado por un filtro de Spring Security en cada petición.

7.2. Modelo de datos

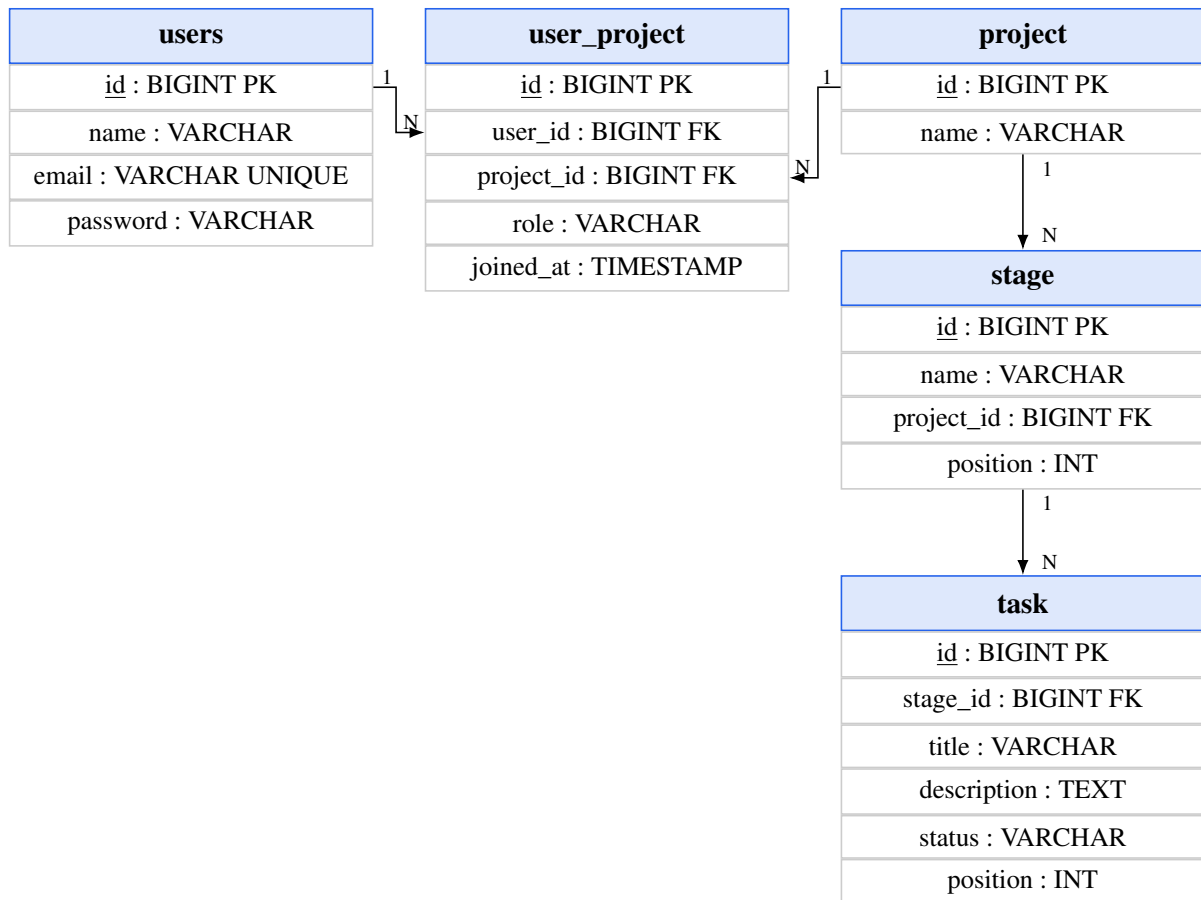


Figura 7.2: Diagrama entidad-relación

Las tablas principales son users, project, user_project (relación muchos a muchos con rol), stage y task. Las contraseñas se almacenan cifradas con **BCrypt** (factor de coste 10), lo que garantiza que no puedan recuperarse aunque la base de datos quede expuesta.

7.3. Diseño de la API REST

La API sigue convenciones REST estándar. Las rutas protegidas requieren la cabecera `Authorization: Bearer <token>` con un token JWT válido.

Tabla 7.1: Endpoints de la API REST

Método	Ruta	Descripción
POST	/users	Registrar usuario
POST	/users/login	Autenticar usuario (devuelve JWT)
GET	/users/{id}	Obtener usuario
PATCH	/users/{id}	Actualizar usuario
DELETE	/users/{id}	Eliminar usuario

Método	Ruta	Descripción
GET	/users/{id}/projects	Proyectos de un usuario
POST	/users/{id}/projects	Crear proyecto (asigna al usuario como OWNER)
GET	/projects	Listar proyectos
GET	/projects/{id}	Obtener proyecto
PATCH	/projects/{id}	Actualizar proyecto
DELETE	/projects/{id}	Eliminar proyecto
GET	/projects/{id}/users	Miembros del proyecto
POST	/projects/{id}/users/{uid}	Añadir miembro
PATCH	/projects/{id}/users/{uid}	Actualizar rol de miembro
DELETE	/projects/{id}/users/{uid}	Eliminar miembro
GET	/projects/{id}/stages	Etapas del proyecto
POST	/projects/{id}/stages	Crear etapa
PATCH	/stages/{id}	Actualizar etapa
DELETE	/stages/{id}	Eliminar etapa
PATCH	/projects/{id}/stages/reorder	Reordenar etapas
GET	/stages/{id}/tasks	Tareas de una etapa
POST	/stages/{id}/tasks	Crear tarea
GET	/tasks/{id}	Obtener tarea
PATCH	/tasks/{id}	Actualizar tarea
PATCH	/tasks/{id}/move	Mover tarea de etapa
DELETE	/tasks/{id}	Eliminar tarea

7.4. Diseño de la interfaz

La aplicación se compone de cuatro vistas principales:

- **Login:** formulario de email y contraseña con validación y mensaje de error.
- **Registro:** formulario reactivo con detección de email duplicado.
- **Dashboard:** cuadrícula de tarjetas de proyecto con formulario inline de creación.
- **Tablero Kanban:** columnas horizontales desplazables con tarjetas de tarea.

La paleta de colores y el espaciado se gestionan mediante variables CSS (*design tokens*) definidas en un archivo central. Esto permite que el modo oscuro se active automáticamente respondiendo a `prefers-color-scheme` sin ningún JavaScript adicional.

Capítulo 8

Implementación

8.1. Stack tecnológico

Tabla 8.1: Stack tecnológico y justificación de las elecciones

Tecnología	Versión	Justificación
Spring Boot	4.0.5	Framework Java maduro, ampliamente usado en la industria
Java	21 LTS	Versión LTS actual con mejoras de rendimiento y streams
Spring Security	7.0	Autenticación y autorización; integración con filtros JWT
JWT	0.12	Generación y validación de tokens JWT con HMAC-SHA256
Spring Data JPA	—	Reduce código de acceso a datos, gestiona el esquema automáticamente
Liquibase	5.x	Versionado del esquema de base de datos
PostgreSQL	16	Base de datos relacional robusta y de código abierto
Docker Compose	—	Orquesta los tres servicios (PostgreSQL, backend y frontend); entorno completo con un comando
JUnit 5 + Mockito	—	Tests unitarios del backend
Angular	20	Framework SPA con signals nativos y arquitectura standalone
TypeScript	5.x	Tipado estático que previene errores en tiempo de desarrollo
Vitest	—	Tests unitarios del frontend
SCSS propio	—	Sin frameworks externos para mayor control del diseño
Angular PWA	—	Service worker y manifiesto para instalación como app de escritorio
springdoc-openapi	2.8	Genera documentación Swagger UI de la API automáticamente; activa solo en perfil dev
Git + GitHub	—	Flujo de trabajo estándar del sector
GitHub Actions	—	CI: tests automáticos y escaneo de secretos en cada push a main
Railway	—	CD del backend: redespiega automáticamente tras cada push aprobado
Vercel	—	CD del frontend: build y publicación en CDN global tras cada push

8.2. Estructura del proyecto

Listing 8.1: Estructura de carpetas del proyecto

```
project-manager/  
+-- backend/                # Spring Boot
```

```

|   +-- src/main/java/
|       +-- controllers/      # UserController, ProjectController...
|       +-- services/        # Lógica de negocio por entidad
|       +-- repositories/    # Interfaces JPA
|       +-- models/          # Entidades JPA
|       +-- dtos/            # Objetos de transferencia de datos
+-- frontend/                # Angular 20
|   +-- src/app/
|       +-- models/          # Interfaces de dominio TypeScript
|       +-- services/        # Servicios HTTP
|       +-- core/            # AuthService, interceptor, guard
|       +-- layout/          # Navbar y shell principal
|       +-- features/
|           |   +-- auth/      # Login y registro
|           |   +-- dashboard/ # Listado de proyectos
|           |   +-- kanban/    # Tablero, columnas, tarjetas
|           |   +-- members/   # Gestión de miembros
|       +-- shared/          # Componentes reutilizables
+-- docker-compose.yml      # Orquestación de la base de datos

```

8.3. Decisiones técnicas destacadas

Signals como capa de presentación sobre Observables

Angular 20 introduce signals como primitiva de reactividad. En este proyecto, todos los componentes exponen signals en lugar de Observables directos, usando `toSignal()` para convertir las respuestas HTTP. Esto simplifica los templates y evita suscripciones manuales.

Carga reactiva de datos con `rxResource`

Para cargar datos del backend de forma reactiva y permitir recargas manuales tras mutaciones (crear etapa, mover tarea...) se usa `rxResource`, la API experimental de Angular 19+ diseñada para este patrón. Declara el parámetro reactivo en `params` y la llamada HTTP en `stream`; Angular reejecutará la carga automáticamente cuando el parámetro cambie, y expone `.reload()` para forzar una recarga manual.

Listing 8.2: Carga reactiva con `rxResource` en `KanbanBoardComponent`

```

private readonly stagesResource = rxResource({
  params: () => this.id(),
  stream: ({ params: id }) => this.stageService.getByProject(id),
});
protected readonly stages = computed(() => this.stagesResource.value()
  ?? []);

private reload(): void {
  this.stagesResource.reload();
}

```

Componentes presentacionales puros

Los componentes de tarjeta (`TaskCardComponent`, `ProjectCardComponent`) son completamente presentacionales: solo reciben datos por `input()` y emiten eventos por `output()`. No

inyectan servicios. Esto facilita su reutilización y prueba.

Diseño responsivo con experiencias diferenciadas

Se tomó la decisión de no limitar la responsividad a redimensionar elementos, sino a ofrecer experiencias distintas según el dispositivo:

- **Escritorio:** todas las columnas visibles en una fila horizontal con scroll; botón de nueva etapa con estilo punteado al final de la fila.
- **Móvil:** una columna por pantalla con *scroll snap* (el usuario desliza de columna en columna); botón de nueva etapa en el encabezado, siempre accesible.

Esta decisión responde a que el patrón de uso es diferente: en escritorio se quiere visión global del tablero, mientras que en móvil se trabaja en una sola columna a la vez.

Orden determinista de tareas en el backend

Las colecciones HashSet de Java no garantizan orden. Esto causaba que las tareas cambiaran de posición en la respuesta de la API al actualizarse. La solución fue añadir un *tiebreaker* por id al comparador de posición en el ProjectMapper:

Listing 8.3: Comparador con tiebreaker para orden determinista

```
private static final Comparator<Task> TASK_ORDER =
    Comparator.comparing(Task::getPosition,
        Comparator.nullsLast(Comparator.naturalOrder())
    )
    .thenComparing(Task::getId);

// En el stream:
.sorted(TASK_ORDER)
```

Documentación de la API con Swagger UI

Se integró springdoc-openapi para generar automáticamente la documentación interactiva de todos los endpoints REST. La interfaz Swagger UI permite explorar y probar la API desde el navegador sin herramientas externas.

Para evitar exponer la documentación en producción, se utiliza el sistema de perfiles de Spring: en application.yaml Swagger está desactivado por defecto, y application-dev.yaml lo habilita. El perfil dev se activa automáticamente en el entorno local mediante docker-compose.yaml, donde se establece la variable SPRING_PROFILES_ACTIVE=dev.

Capítulo 9

Pruebas

9.1. Pruebas unitarias del backend

El backend incluye tests unitarios con JUnit 5 y Mockito para los servicios principales.

Tabla 9.1: Cobertura de pruebas unitarias del backend (105 tests, 100% cobertura)

Clase	Casos cubiertos	Tests	Resultado
UserServiceTest	Crear, login, email duplicado, no encontrado, eliminar	15	✓
UserControllerTest	Endpoints con JWT mockeado e identidad verificada	14	✓
TaskServiceTest	Crear, mover entre proyectos, paths not-found	14	✓
ProjectMemberServiceTest	Añadir, actualizar rol, validación de rol, permisos	15	✓
StageServiceTest	Crear, reordenar, renombrar, eliminar, no encontrado	12	✓
ProjectServiceTest	Crear, obtener miembros, eliminar proyecto	6	✓
TaskControllerTest	Todos los endpoints con contexto JWT mockeado	6	✓
StageControllerTest	Todos los endpoints con contexto JWT mockeado	5	✓
ProjectControllerTest	Todos los endpoints con contexto JWT mockeado	5	✓
ProjectMemberControllerTest	Añadir, actualizar y eliminar miembro	5	✓
ProjectPermissionSvcTest	Verificar roles OWNER, ADMIN y MEMBER	3	✓
ProjectMapperTest	Mapeo de entidades con streams y comparadores	2	✓
UserMapperTest	Mapeo de entidades y colecciones de proyecto	2	✓
BackendApplicationTests	Carga de contexto Spring	1	✓
Total		105	✓

Cobertura con JaCoCo

La cobertura se mide con el plugin JaCoCo integrado en el ciclo verify de Maven. Se excluyen del informe las clases sin lógica de negocio (entity/**, model/**, config/**,

repository/** y BackendApplication). Sobre el código de servicio, controlador y mapper se obtiene el **100 %** de cobertura de instrucciones y ramas.

9.2. Pruebas unitarias del frontend

El frontend incluye 16 tests unitarios con **Vitest**. Cubren los servicios y utilidades del núcleo de la aplicación.

Tabla 9.2: Pruebas unitarias del frontend (Vitest)

Clase	Casos cubiertos	Resultado
AuthService	Sesión, token JWT, login/logout, persistencia en localStorage	✓
ToastService	Añadir, eliminar y expirar notificaciones toast	✓
authGuard	Redirección al login si no autenticado	✓
authInterceptor	Inyección de cabecera Authorization en peticiones autenticadas	✓

9.3. Integración y despliegue continuos

El proyecto cuenta con pipelines de CI y CD en distintas plataformas:

Integración continua — GitHub Actions

- **ci.yml**: ejecuta los tests del frontend (`npx vitest run`) y del backend (`./mvnw verify` con JaCoCo) en cada push y pull request sobre main.
- **gitleaks.yml**: escanea el repositorio en busca de secretos y credenciales en cada push.

Despliegue continuo

- **Railway** (backend + PostgreSQL): detecta cada push a main y redespiega automáticamente el servidor Spring Boot. La base de datos PostgreSQL 16 está gestionada como servicio propio dentro del mismo proyecto de Railway.
- **Vercel** (frontend): detecta cada push a main, ejecuta el build de Angular (`ng build`) y publica el resultado como sitio estático en su CDN global.

De este modo, cualquier cambio mergeado a main pasa por los tests automáticos y, si se superan, queda desplegado en producción sin intervención manual.

9.4. Pruebas funcionales del frontend

Tabla 9.3: Pruebas funcionales del frontend

Caso	Pasos	Resultado esperado	Estado
Login correcto	Email y contraseña válidos	Redirige al dashboard	✓
Login incorrecto	Contraseña errónea	Mensaje de error	✓
Email duplicado	Registrar email existente	Mensaje de error	✓
Crear proyecto	Rellenar nombre y confirmar	Aparece en el dashboard	✓
Crear etapa	Pulsar «Añadir etapa»	Nueva columna visible	✓
Crear tarea	Pulsar «Añadir tarea»	Tarea en la columna	✓
Mover tarea	Cambiar columna en modal	Tarea en nueva columna	✓
Editar tarea	Abrir modal, modificar y guardar	Cambios en la tarjeta	✓
Eliminar tarea	Confirmar en diálogo	Tarea eliminada de la columna	✓
Modo oscuro	Sistema en dark mode	Toda la UI cambia de tema	✓
Responsivo móvil	Viewport 390px	Columnas en scroll snap	✓
Toast confirmación	Guardar o eliminar tarea	Notificación visible	✓

Capítulo 10

Manual de despliegue

10.1. Requisitos previos

Tabla 10.1: Requisitos de software para el despliegue

Software	Versión mínima
Java JDK	21
Node.js	20 LTS
Docker	Cualquier versión reciente

10.2. Con Docker (recomendado)

El proyecto incluye un `docker-compose.yml` que orquesta los tres servicios: PostgreSQL, backend Spring Boot y frontend Angular. Con un único comando se levanta el entorno completo:

Listing 10.1: Levantar el stack completo con Docker Compose

```
docker compose up -d
```

Cada servicio tiene su `Dockerfile` con compilación multietapa. El frontend se construye con Node y se sirve con nginx; el backend con Maven y se ejecuta sobre JRE 21. La aplicación queda disponible en `http://localhost:4200`, la API en `http://localhost:8080` y la base de datos en el puerto 5432.

10.3. Sin Docker

Base de datos

Listing 10.2: Levantar solo PostgreSQL

```
docker compose up db -d
```

Backend

Listing 10.3: Arrancar el servidor Spring Boot

```
cd backend
./mvnw spring-boot:run
```

El servidor arranca en `http://localhost:8080`. Spring Boot crea o actualiza el esquema automáticamente al arrancar (`ddl-auto: update`).

10.4. Frontend

Listing 10.4: Arrancar el servidor de desarrollo Angular

```
cd frontend  
npm install  
npm run serve
```

La aplicación queda disponible en `http://localhost:4200`.

Capítulo 11

Manual de usuario

11.1. Registro e inicio de sesión

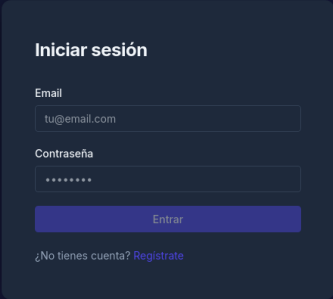
A dark-themed login form titled "Iniciar sesión" is centered on a dark blue background. The form contains two input fields: "Email" with the placeholder text "tu@email.com" and "Contraseña" with masked characters "*****". Below the password field is a blue "Entrar" button. At the bottom of the form, there is a link that reads "¿No tienes cuenta? [Regístrate](#)".

Figura 11.1: Formulario de login. Elaboración propia.

Al acceder a la aplicación se muestra el formulario de login. Si no se tiene cuenta, el enlace «Regístrate» lleva al formulario de registro, donde hay que introducir nombre, email y contraseña.

11.2. Dashboard



Figura 11.2: Dashboard con lista de proyectos. Elaboración propia.

Tras iniciar sesión se accede al dashboard, donde se muestran todos los proyectos del usuario. Se puede crear un nuevo proyecto con el botón «Nuevo proyecto» y eliminar uno existente con el botón de papelera de su tarjeta.

11.3. Tablero Kanban

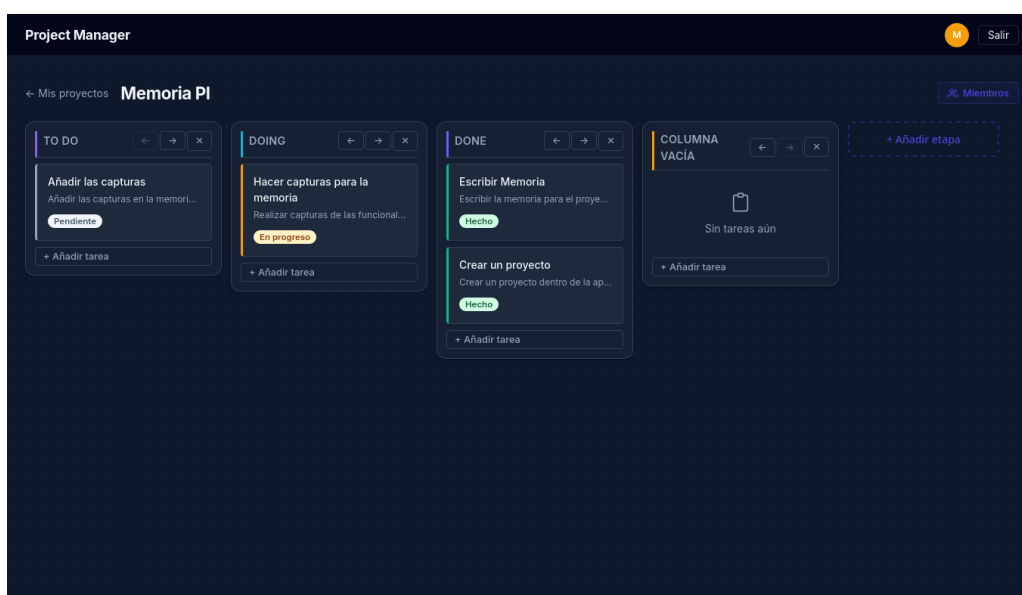


Figura 11.3: Tablero Kanban con columnas y tareas. Elaboración propia.

Al pulsar sobre un proyecto se abre el tablero. Las columnas representan las etapas del flujo de trabajo. Con el botón «Añadir etapa» (o el botón punteado en escritorio) se crea una nueva

columna. Con «Añadir tarea» dentro de cada columna se crea una tarea. Para editar o mover una tarea a otra etapa se pulsa sobre su tarjeta para abrir el modal de edición.

11.4. Gestión de miembros

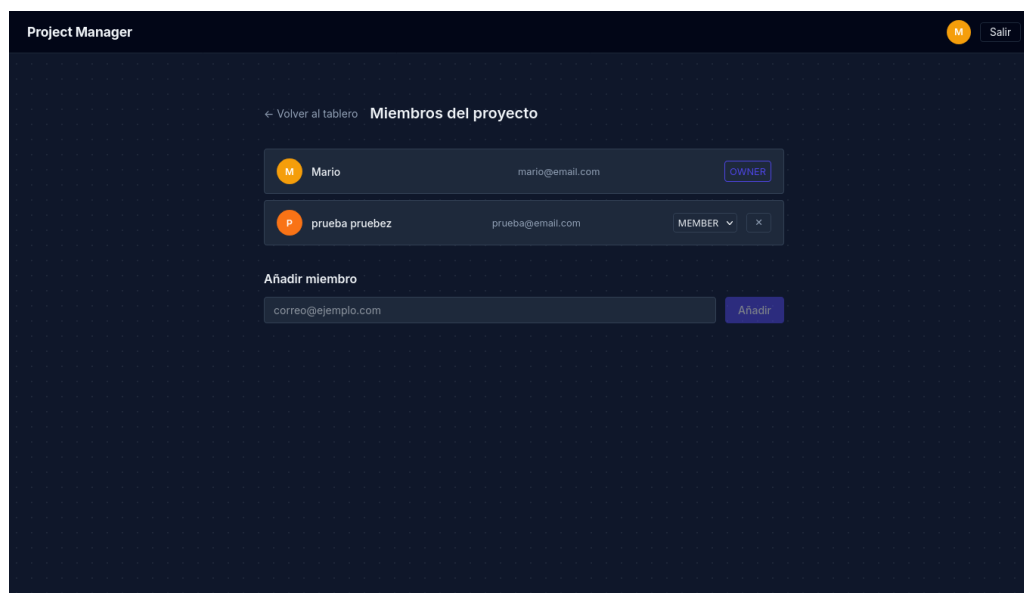


Figura 11.4: Página de miembros del proyecto. Elaboración propia.

Desde el botón «Miembros» del tablero se accede a la página de gestión de miembros. Aquí se pueden añadir nuevos usuarios al proyecto asignándoles un rol (propietario, editor o lector) y eliminar miembros existentes.

Capítulo 12

Análisis del producto (FOL)

12.1. Descripción del proyecto como producto

Project Manager es una aplicación web colaborativa de gestión de proyectos y tareas basada en metodología Kanban. Permite a equipos de trabajo organizar sus proyectos visualmente mediante tableros con columnas (etapas) y tarjetas (tareas), gestionando quién hace qué y en qué estado se encuentra cada tarea en todo momento.

Usuario objetivo

El producto está orientado a tres perfiles principales:

- **Equipos pequeños y medianos** (2–15 personas) que necesitan una herramienta ligera sin la complejidad de soluciones enterprise como Jira.
- **Freelancers y profesionales independientes** que gestionan múltiples proyectos simultáneamente.
- **Estudiantes y grupos de trabajo académico** que buscan coordinarse sin depender de hojas de cálculo compartidas.

Modelo de entrega

Aplicación web accesible desde cualquier navegador moderno, sin instalación en el cliente. El backend expone una API REST y el frontend es una SPA. Podría desplegarse en la nube (p. ej. con el backend en Railway y el frontend en Vercel) con coste mínimo o nulo en fases iniciales.

12.2. Análisis DAFO

Tabla 12.1: Análisis DAFO del producto

Fortalezas ■ Tecnología moderna: Angular 20 con signals, Spring Boot con Java 21 ■ Principios SOLID y clean code; código fácil de mantener y extender ■ Interfaz cuidada: dark/light mode, responsiva, animaciones sutiles ■ Sin dependencias de frameworks UI; control total del diseño ■ API REST desacoplada; reutilizable con una app móvil futura ■ Despliegue sencillo con Docker Compose	Oportunidades ■ Nicho en equipos pequeños que perciben Jira/Asana como excesivos ■ Crecimiento del trabajo remoto y demanda de herramientas de coordinación asíncrona ■ Mercado educativo: equipos de clase sin presupuesto para licencias ■ Diferenciación mediante self-hosting y código abierto ■ Integraciones futuras (GitHub, Slack) como valor añadido sin cambiar el núcleo
Debilidades ■ Sin drag & drop; mover tareas con selector desplegable es menos intuitivo que arrastrar ■ Sin notificaciones en tiempo real; los cambios de otros usuarios requieren recargar la página ■ Sin estrategia de copias de seguridad de la base de datos ■ Despliegue manual; sin pipeline de entrega continua a producción	Amenazas ■ Mercado muy saturado: Trello, Jira, Asana, Notion, Linear y Monday.com tienen años de ventaja ■ Efecto red: los equipos adoptan la herramienta que ya usa la mayoría ■ Dependencia de planes gratuitos de plataformas cloud cuyos términos pueden cambiar ■ Angular publica versiones mayores frecuentemente; mantener el proyecto actualizado requiere dedicación continua

12.3. Análisis de mercado

Panorama competitivo

El mercado de gestión de proyectos está dominado por herramientas SaaS con modelos freemium. La tabla 12.2 recoge las principales referencias del sector.

Tabla 12.2: Comparativa de competidores en el mercado

Herramienta	Fortaleza principal	Debilidad principal	Precio base
Trello	Simplicidad, Kanban puro	Sin subtareas ni informes	Gratis / 5\$/mes
Asana	Vistas múltiples, dependencias	Complejo para equipos pequeños	Gratis / 10\$/mes
Jira	Configurable, sprints, GitHub	Lento y complejo	Gratis / 7\$/mes
Notion	Todo-en-uno (docs + tareas)	No especializado en proyectos	Gratis / 8\$/mes
Linear	Rápido, excelente UX técnica	Solo para desarrollo software	8\$/mes
Monday.com	Visualizaciones avanzadas	Sin plan gratuito permanente	9\$/mes

Segmento objetivo y posicionamiento

El producto encaja mejor en el segmento de Trello: usuarios que quieren la claridad visual del Kanban sin la complejidad de Jira ni el coste de Monday. La diferenciación potencial se apoya en tres ejes:

- **Self-hosting:** el usuario despliega la aplicación en su propio servidor, con control total de sus datos.
- **Sin límites artificiales:** en un modelo open source no hay restricciones de número de tableros o miembros.
- **Personalización:** al ser código abierto, los equipos técnicos pueden adaptar la herramienta a sus procesos.

Tendencias del mercado

- **Consolidación all-in-one:** Notion y Linear integran documentación y tareas. Una hoja de ruta que incluya wikis por proyecto respondería a esta tendencia.
- **IA en herramientas de productividad:** Asana, Notion y Linear han incorporado asistentes de IA. Es una expectativa creciente en todos los segmentos.
- **Mobile-first:** el uso de herramientas de productividad desde el móvil ha crecido significativamente. El diseño con *scroll snap* columna a columna del proyecto responde directamente a esta tendencia.

12.4. Propuesta de mejora y evolución futura

Tabla 12.3: Hoja de ruta de mejoras del producto

Horizonte	Mejora	Justificación
Corto plazo	Drag & drop de tareas	Es la interacción más esperada en un tablero Kanban
Corto plazo	Autenticación mediante cookies HTTP-only	El token JWT se almacena actualmente en <code>localStorage</code> , que algunos navegadores limpian al cerrarse. Moverlo a una cookie <code>HttpOnly</code> con fecha de expiración explícita resolvería este comportamiento y añadiría protección frente a ataques XSS, ya que las cookies <code>HttpOnly</code> no son accesibles desde JavaScript
Corto plazo	Sistema de invitaciones a proyectos	Actualmente los administradores añaden miembros de forma directa; sería más adecuado que el usuario destinatario recibiera una invitación y la aceptara o rechazara, mejorando la privacidad y el control sobre la pertenencia al proyecto
Corto plazo	Transferencia de propiedad de proyecto	Actualmente el propietario (<i>OWNER</i>) no puede abandonar un proyecto; solo puede eliminarlo. Permitir transferir el rol de <i>OWNER</i> a otro miembro daría más flexibilidad en la gestión de equipos
Corto plazo	Despliegue en la nube	Railway (backend + PostgreSQL) + Vercel (frontend) permitirían una demo pública sin coste inicial
Medio plazo	Notificaciones en tiempo real (WebSockets)	Los cambios de otros miembros deben reflejarse sin recargar
Medio plazo	Asignación de tareas a miembros	El modelo de datos ya lo contempla; falta exponerlo en la UI
Medio plazo	Fechas límite y vista de calendario	Añade valor para la planificación sin cambiar la arquitectura
Largo plazo	App móvil nativa (Capacitor)	La API REST ya está lista; reutilizar el backend reduciría el coste
Largo plazo	Integraciones externas (GitHub, Slack)	Diferenciador frente a competidores y encaje en flujos existentes

Capítulo 13

Conclusiones

13.1. Grado de consecución de objetivos

El proyecto ha cumplido todos los objetivos planificados. El backend REST está completamente funcional, con una API que cubre todas las operaciones necesarias para gestionar proyectos, etapas, tareas y miembros. El frontend Angular opera de forma fluida con las funcionalidades clave del tablero Kanban, incluyendo la interfaz responsiva con soporte de modo claro y oscuro.

La arquitectura desacoplada (SPA + API REST) facilita el mantenimiento y la evolución independiente de cada capa, y el código sigue principios SOLID y clean code en todo el proyecto.

13.2. Problemas encontrados

- **Reactividad en Angular 20:** la API de signals es reciente y la documentación sobre patrones avanzados de carga reactiva es escasa. Se evaluaron soluciones manuales (signal + toObservable + switchMap) antes de adoptar rxResource, la API oficial de Angular para este caso de uso.
- **Orden no determinista de tareas:** HashSet no garantiza orden; las tareas cambiaban de posición aleatoriamente al actualizarse. Se resolvió añadiendo un *tiebreaker* por id en el comparador del ProjectMapper.
- **Modo oscuro en controles nativos:** los elementos select del navegador no se adaptaban al modo oscuro. La solución fue añadir la propiedad CSS color-scheme: dark a la hoja de tokens, instruyendo al navegador para que renderice todos los controles nativos en el tema correcto.
- **Configuración de Spring Security:** desactivar la autoconfiguración del servicio de usuarios vía YAML resultó no fiable en ciertos contextos de arranque. La solución fue declarar la exclusión directamente en @SpringBootApplication, que se procesa antes de que el autoconfigurador llegue a registrarse.

13.3. Valoración personal

Este proyecto ha permitido integrar en un producto real todos los conocimientos del ciclo: diseño de base de datos relacional, desarrollo de API REST con Spring Boot, construcción de una SPA con Angular y maquetación CSS avanzada. Las dificultades encontradas, lejos de ser obstáculos, han sido las que más aprendizaje han generado, especialmente en lo relativo a la reactividad de Angular 20 y al diseño de experiencias de usuario responsivas diferenciadas por dispositivo.

Capítulo 14

Repositorio y software utilizado

14.1. Repositorio GitHub

URL: <https://github.com/mapacheexe/project-manager>

Tabla 14.1: Estructura del repositorio

Ruta	Contenido
backend/	Código fuente del servidor Spring Boot
backend/src/main/resources/application.yaml	Configuración del servidor y base de datos
backend/pom.xml	Dependencias Maven
frontend/	Código fuente de la SPA Angular
frontend/src/app/models/	Interfaces TypeScript de dominio
frontend/src/app/services/	Servicios HTTP
frontend/src/app/core/	AuthService, interceptor, guard
frontend/src/app/features/	Componentes organizados por funcionalidad
frontend/src/app/shared/	Componentes reutilizables (modal, toast, confirm)
frontend/src/styles/	Parciales SCSS: tokens, botones, modales, reset
docker-compose.yml	Orquestación de los tres servicios (BD, backend, frontend)
backend/Dockerfile	Imagen Docker del servidor Spring Boot
frontend/Dockerfile	Imagen Docker del frontend (build Node + nginx)
frontend/nginx.conf	Configuración nginx para SPA (re-dirige rutas a index.html)

14.2. Software utilizado

Tabla 14.2: Software utilizado en el desarrollo

Software	Versión	Uso
IntelliJ IDEA	2024	IDE principal para backend y frontend
Java JDK	21	Compilación y ejecución del backend
Apache Maven	3.9	Gestión de dependencias del backend
Spring Boot	4.0.5	Framework del servidor REST
Node.js	20 LTS	Entorno de ejecución para Angular CLI y npm
Angular CLI	20	Herramienta de desarrollo y build del frontend
TypeScript	5.x	Lenguaje del frontend
Docker Desktop	—	Contenerización de PostgreSQL
PostgreSQL	16	Base de datos relacional
Git	2.x	Control de versiones
GitHub	—	Repositorio remoto y publicación
Postman	—	Pruebas manuales de la API REST

Bibliografía

- Angular. *Angular Documentation*. Google LLC, 2024. Consultado el 10/05/2026.
<https://angular.dev>
- Angular. *Angular Signals Guide*. Google LLC, 2024. Consultado el 12/05/2026.
<https://angular.dev/guide/signals>
- Microsoft. *TypeScript Documentation*. Microsoft, 2024. Consultado el 08/05/2026.
<https://www.typescriptlang.org/docs>
- ReactiveX. *RxJS Documentation*. ReactiveX, 2024. Consultado el 10/05/2026.
<https://rxjs.dev>
- Vitest. *Vitest Documentation*. 2024. Consultado el 22/05/2026.
<https://vitest.dev>
- Pivotal Software. *Spring Boot Reference Documentation*. VMware, 2024. Consultado el 15/03/2026.
<https://spring.io/projects/spring-boot>
- Spring. *Spring Security Reference Documentation*. VMware, 2024. Consultado el 20/05/2026.
<https://docs.spring.io/spring-security/reference>
- Jones, M.; Bradley, J.; Sakimura, N. *JSON Web Token (JWT)*. RFC 7519. IETF, 2015. Consultado el 20/05/2026.
<https://datatracker.ietf.org/doc/html/rfc7519>
- jwtk. *JJWT — Java JWT Library*. 2024. Consultado el 21/05/2026.
<https://github.com/jwtk/jjwt>
- Spring. *Spring Data JPA Reference Documentation*. VMware, 2024. Consultado el 15/03/2026.
<https://docs.spring.io/spring-data/jpa/reference>
- JUnit Team. *JUnit 5 User Guide*. 2024. Consultado el 22/05/2026.
<https://junit.org/junit5/docs/current/user-guide>
- Mockito. *Mockito Framework Documentation*. 2024. Consultado el 22/05/2026.
<https://site.mockito.org>
- Liquibase. *Liquibase Documentation*. Liquibase, 2024. Consultado el 21/05/2026.
<https://docs.liquibase.com>
- The PostgreSQL Global Development Group. *PostgreSQL 16 Documentation*. 2024. Consultado el 20/03/2026.
<https://www.postgresql.org/docs/16>
- Docker Inc. *Docker Compose Documentation*. 2024. Consultado el 18/03/2026.
<https://docs.docker.com/compose>

- GitHub. *GitHub Actions Documentation*. GitHub, 2024. Consultado el 22/05/2026.
<https://docs.github.com/en/actions>
- MDN Web Docs. *CSS Scroll Snap*. Mozilla, 2024. Consultado el 05/05/2026.
https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_scroll_snap
- MDN Web Docs. *CSS color-scheme property*. Mozilla, 2024. Consultado el 08/05/2026.
<https://developer.mozilla.org/en-US/docs/Web/CSS/color-scheme>
- Verou, Lea. *CSS3 Patterns Gallery*. 2011. Consultado el 19/05/2026.
<https://projects.verou.me/css3patterns/>
- Martin, Robert C. *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall, 2008.